CADT IDT

W1 PRACTICE

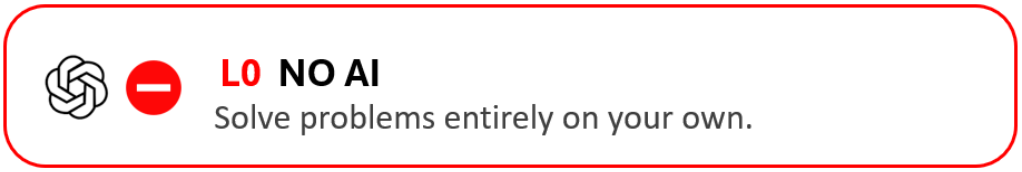
PART 2 – MANIPULATION

⚠ Before this practice

- You need to have completed: `PART 1- EXPLORATION`

What do you need to **be able to do** for next week?

- Identify the **type** of variable
- Manipulate **nullable** and **non-nullable** variables.
- Differentiate between **final** and **const**.
- Manipulate **strings, lists, and maps.**
- Use **loops** and **conditions** to control flow.
- Define and call functions with **positional** and **named arguments**, understand **arrow syntax**



Are you lost?

Read the following documentation to be ready for this practice:

- ✓ Variables
- ✓ Null Safety
- ✓ Built-in types
- ✓ Lists
- ✓ Loops
- ✓ Conditions
- ✓ Functions



*EX 1 – Manipulate **Types***

Are you clear about strings, integer, list, map, set, objects in Dart?
Examine the given data structures and **write the inferred Dart type** for each one (*see example*)

Notes

- *First find by yourself the type*
- *If you need, double check your answer with VSCode.*

Data	Dart Type

`const studentGrades = {`
`  'Sokan': [90, 85, 88],`
`  'Sokea': [70, 80, 75],`
`  'Hay': [95, 92, 89],`
`};`
	`Map<String, List<int>>`

`const pizzaPrices = {`
`  'margherita': 5.5,`
`  'pepperoni': 7.5,`
`  'vegetarian': 6.5,`
`};`
	`Map<String, double>`

`const books = [`
`  {'title': '1984', 'author': 'George Orwell'},`
`  {'title': 'Brave New World', 'author': 'Aldous Huxley'},`
`  {'title': 'Fahrenheit 451', 'author': 'Ray Bradbury'},`
`];`
	`List<Map<String, String>>`

`const company = {`
`  'HR': {'employees': 5, 'budget': 20000},`
`  'IT': {'employees': 10, 'budget': 50000},`
`  'Marketing': {'employees': 7, 'budget': 30000},`
`};`
	`Map<String, Map<String, int>>`

`const coordinates = [`
`  [1, 2],`
`  [3, 4],`
`  [5, 6],`
`];`
	`List<List<int>>`

`const productDetails = {`
`  'id': 101,`
`  'name': 'Laptop',`
`  'price': 999.99,`
`  'inStock': true,`
`};`
	`Map<String, object>`

`const operations = [`
`  (int a, int b) => a + b,`
`  (int a, int b) => a - b,`
`  (int a, int b) => a * b,`
`];`
	`double function(int, int)`

`const distances = {3.1, 5.5, 10.2, 7.8};`	`set<double>`

`const university = {`
`  'departments': [`
`  {`
`  'name': 'Computer Science',`
`  'students': [`
`  {'name': 'Alice', 'age': 22},`
`  {'name': 'Bob', 'age': 24},`
`  ]`
`  },`
`  {`
`  'name': 'Mathematics',`
`  'students': [`
`  {'name': 'Charlie', 'age': 21},`
`  {'name': 'Dave', 'age': 23},`
`  ]`
	`Map<String, List<Map<String, Object>>>`

```
    }
  ]
};
```

*EX 2 – Manipulate **final** and **const***

In this exercise, you need to decide which variable can be declared as **const** or **final**.

```dart
// 1 - startText
String iLike = 'I like pizza';

// 2 - toppings
String toppings = 'with tomatoes';
toppings += " and cheese";

// 3 - message
String message = '$iLike $toppings';
print(message);

// 4 - rbgColors
List<String> rbgColors = ['red', 'blue', 'green'];
print(rbgColors);

// 5 - weekDays
List<String> weekDays = ['monday', 'tuesday', 'wednesday'];
weekDays.add('thursday');
print(weekDays);

// 6 - scores
List<int> scores = [45,35,50];
scores = [45,35,50, 78];
print(scores);
```

Guess which variables can be declared as **const**, **final** or **var**, and explain your choices.

Notes

- *Read here to understand the concepts.*
- *Prefer const over final over var.*

	VAR, FINAL, CONST?	**WHY**
`iLike`	CONST	*Because this variable never changes*
`toppings`	VAR	*Because the client might want to change their toping or add more.*
`message`	FINAL	*Because it is fixed to call on 2 other variables.*
`rbgColors`	CONST	*Because RGB only consists of those 3 colors.*
`weekDays`	VAR	*Because weekdays are to be adjusted occasionally.*
`score`	CONST	*Because score must not be modified, but they can still be added.*

*EX 3 – **Filter** a list*

Write a Dart program that takes a list of exam scores and **returns the scores of all students who passed**

Given

- You are given a list of integers representing students' exam scores.
- A student **passes** if their score is **50 or higher**.

Example input:

```
[45, 78, 62, 49, 85, 33, 90, 50]
```

Example output:

```
[78, 62, 85, 90, 50]
```

Requirements

Implement the filtering in **two different ways**.

Method 1 — Using a for loop

- Use a for loop to examine each score.
- Add passing scores (>= 50) to a new list.

Method 2 — Using where()

- Use **where** function to filter the list.
- Pass a **proper anonymous function** to where().

```
void main() {
  List<int> scores = [45, 78, 62, 49, 85, 33, 90, 50];
  // Method 1
  List<int> result = [];
  for(int i = 0; i < scores.length; i++) {
    if(scores[i] >= 50) {
      result.add(scores[i]);
    }
  }
  print(result);

  // Method 2
  List<int> result2 = scores.where((s) => (s >= 50)).toList();
  print(result2);
  }
```